Team 4 ROB 550 BotLab Report

Tzu-Chieh Tai, Hao Duan, Aidan Dempster

***Abstract*—This report details the development and integration of a full autonomy stack for a differential-drive mobile robot, the MBot, designed to perform exploration and object manipulation tasks. The project integrates a custom-designed scissor-lift mechanism for stacking AprilTag-labeled crates with a robust software architecture for navigation. The system employs a probabilistic occupancy grid for mapping and Monte Carlo Localization (MCL) utilizing a likelihood field model for state estimation. Path planning is executed via an A* algorithm augmented with a safety-aware cost function that penalizes proximity to obstacles, while trajectory tracking is handled by a hybrid pure pursuit and turn-in-place controller. A frontier-based exploration strategy is implemented to enable autonomous map construction. Experimental results demonstrate that a particle count of 800 and a map resolution of 0.01 m/pixel provide the optimal balance between computational efficiency and localization accuracy. Furthermore, the implementation of a scan buffer significantly reduced mapping artifacts caused by transformation delays. The final integrated system successfully demonstrated autonomous navigation, precise localization, and reliable material handling in a structured environment.**

I. Introduction

MOBILE robots are robotic systems capable of autonomously navigating within an environment[1]. Supported by perception, localization, planning, and control modules, they are able to move freely and adjust their spatial position as required by the task.

From the perspective of locomotion, mobile robots can be classified according to the mechanisms they use to move through their environment[2]. The most common types are wheeled robots, which move in a 2D plane. Legged robots, inspired by biological locomotion, offer superior mobility over obstacles and irregular surfaces. Beyond ground robots, UAVs achieve movement through propellers and can operate in three-dimensional space. Among all locomotion types, wheeled robots are by far the most widely used in real-world applications due to their stability, high energy efficiency[3], and suitability for structured environments such as warehouses, factories, and indoor service settings.

In this project, we integrated control, mapping, localization, and planning modules into a two-wheeled differential-drive robot known as Mbot[4]. We also designed a forklift attachment that enables the robot to stack AprilTag-labeled crates to simulate typical warehouse operations.

To enable autonomy, the robot employs several core algorithms for mapping, localization, planning, and control. Mapping is performed using a probabilistic occupancy grid[5], where each cell maintains a log-odds estimate of occupancy that is updated incrementally from laser measurements. Localization is achieved through Monte Carlo Localization (MCL)[6], which represents the robot's pose uncertainty using a set of weighted particles and updates them through action and sensor models. Path planning is handled by an A* algorithm[7] augmented with a safety-aware cost function that penalizes states near obstacles, encouraging paths with greater clearance. The resulting paths are executed using a pure pursuit controller[8], which selects a lookahead goal point along the path and computes the curvature of the circular arc connecting the robot to that point, generating smooth linear and angular velocity commands for real-time tracking. Additionally, a frontier-based exploration strategy[9] enables the robot to identify unexplored regions of the occupancy grid and autonomously navigate toward them, allowing it to progressively build a complete map of the environment.

The remainder of this report is organized as follows. Section II details the methodology behind each subsystem, including motion control, SLAM, path planning, and exploration. Section III presents the experimental results and discussion, highlighting controller performance, mapping and localization accuracy, and the effects of the safety-aware planning cost function. Section IV concludes the report with a summary of findings and key takeaways.

II. Methodology

A. Motion Controller

We use a hybrid state machine that has two main states: `PURSUIT` and `TURNING`. Additionally, we have an `IDLE` state which is active when there is no planned path and a `FINAL_TURN` that is active after the bot has reached the goal and is aligning with the final waypoint's direction.

Pure Pursuit: In a pure pursuit controller, we treat the path as a set of line segments and continuously drive in circular arcs that lie tangent to the robot direction and intersect with a target point ahead on the path. To compute the target point, we find the intersections between a circle centered on the robot and the path

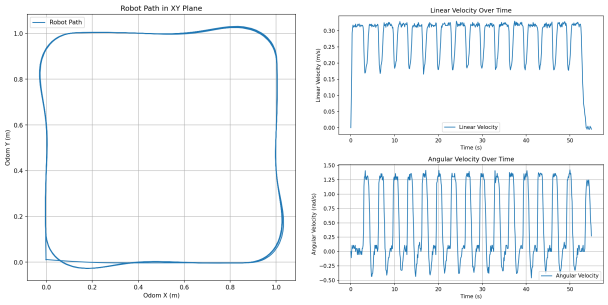


Fig. 1: Odometry measurements when driving a 1m square 4 times. Left position right velocity.

segments and choose the one closest to and just after the previous target point. If no such point is found, we use the previous target point.

Turning: If the angle between the bot forward and the target point exceeds `turn_start_angle` degrees, we swap to `TURNING` state where we turn in place using a proportional control rule until we are within `turn_stop_angle` degrees of the direction to the target point. This allows the robot to recover from sharp turns that would normally cause the pure pursuit controller to fail. This is especially evident at 180 degree turns where the pure pursuit turn radius goes to infinity and the robot drives straight forward instead of turning around.

B. Simultaneous Localization and Mapping (SLAM)

1) Mapping: Mapping is mainly about constructing a map based on measurements received by the robot. The map is updated through a sensor model, which infers the state of a specific point or grid cell using predefined parameters. In practice, before feeding scans into the sensor model, each beam's origin and orientation is interpolated using the `MovingLaserScan()` function to account for robot movement during scanning.

2) Monte Carlo Localization: In our SLAM system, we adopt Monte Carlo Localization (MCL) to estimate the robot's position. Instead of maintaining a single pose estimate with covariance, MCL represents the belief using a set of particles, where each particle denotes a possible robot configuration and is assigned a weight corresponding to its likelihood. This particle-based representation can be viewed as an approximation of the convolution operation in a Bayes Filter.

The filter consists of two stages: a prediction step, governed by the action model, and a correction (or measurement) step, governed by the sensor model. At each iteration, particles are first propagated through the action model, spreading them across potential poses based on robot motion. Then, during the correction step, particle observations are compared against real sensor measurements. Particles whose predicted observations better match the measurements receive higher weights, making them more likely to persist through resampling and converge toward the true robot pose.

a) Action Model: The objective of the action model is to predict the robot's pose given the prior state and the issued motion command. Particles serve as an approximation of the convolution operation in the Bayes Filter. To account for real-world noise, the motion range is scaled by a rotational factor k_1, translational factor k_2, helping the particle spread better reflect the robot's actual pose.

b) Sensor Model: The sensor model estimates the probability of receiving a measurement given the map, and two common approaches are the beam model and the likelihood field model. The beam model simulates ray casting from each particle and evaluates likelihood based on the scan, offering high accuracy but with a runtime complexity of $\mathcal{O}(M\,N\,R)$, where M is the number of particles, N is the number of beams per scan, and R is the maximum beam length.

The likelihood field model instead uses a precomputed obstacle distance map, storing at each grid cell the distance to the nearest obstacle. Measurement evaluation is then performed by checking only the beam endpoint. This reduces computation significantly: building the distance map costs $\mathcal{O}(L)$ and runtime evaluation scales as $\mathcal{O}(M\,N)$, where L is the number of grid cells. Although less precise than the beam model, its approximation is sufficient for practical use.

In our SLAM system, we adopt the likelihood field model. The likelihood of a measurement given the map is modeled using a Gaussian distribution:

$$p(z \mid x, m) = \eta \exp\left(-\frac{d^2}{2\sigma^2}\right),$$

where d is the distance from the beam endpoint to the nearest obstacle in the distance map, σ is the Gaussian noise parameter, and η is the normalization constant. A higher likelihood is assigned when the endpoint is closer to an obstacle, which naturally incorporates measurement noise through the Gaussian formulation.

c) Particle Filter: The particle filter combines the action and sensor models by maintaining a set of weighted particles, where each weight reflects the likelihood of the robot being at that pose. Upon receiving a new scan, the current particles act as the prior. They are first propagated through the action model to generate possible poses, then evaluated by the sensor model to down-weight or discard those inconsistent with the measurements. Resampling produces a new particle set representing the posterior.

The final robot pose is estimated by averaging particle positions, while orientation is computed from the mean of the Cartesian components of particle headings.

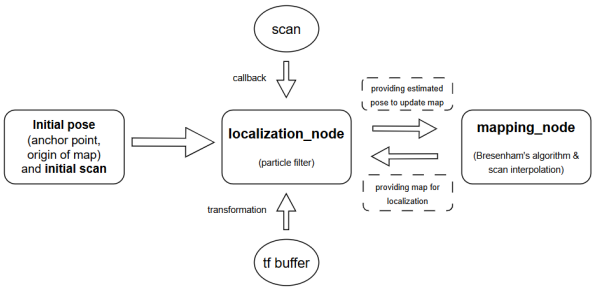


Fig. 2: Workflow of SLAM

3) Combined Implementation: With the mapping algorithm and particle filter working together, as the workflow shown in the Fig. 2, the robot is able to construct a map while simultaneously localizing itself within it. The map origin is typically used as the anchor reference for the environment, and the map is incrementally updated based on this anchor and the initial sensor scans. Using the current map and odometry data, the robot can determine its pose relative to the map origin, allowing the environment to be gradually and continuously expanded. Importantly, the map is updated using the estimated poses from the particle filter, rather than relying solely on raw odometry or sensor readings.

C. Planning and Exploration

1) Path Planning: Given a map, the task of path planning is to define a set of waypoints starting from the current robot state and ending at the goal state that are drivable by the robot. Path planning can be divided into two main approaches: search-based methods and sampling-based methods. In search-based methods the configuration state is discretized into a grid where each cell is a node. Graph search methods can then be used to find the optimal path through the graph. Sampling-based methods instead directly sample in continuous space and use randomness to progressively explore until a path is found.

In mobile robot navigation, search-based algorithms are more popular. Since mobile robots usually navigate on the ground, it is suitable for these algorithms to search in the three dimensional x-y-theta space. Hence, in our project, we implemented the A* algorithm which is search-based for our robot navigation.

A* uses a heuristic to minimize the number of nodes necessary to expand before the optimal path to the goal state is found. It is structurally similar to breadth first search, but instead of popping the node closest to the start point from the queue, we use pop the node with highest priority as defined by:

$$f(n) = g(n) + h(n) \tag{1}$$

where $f(n)$ is the estimated total cost of path through node n to the goal, $g(n)$ is the cost so far to reach node n and $h(n)$ is the estimated cost from node n to the goal.

A* is guaranteed to find the least-cost path as long as the heuristic function overestimates the true cost of traveling from node n to the goal state, a condition called admissibility. Since our robot is free to travel in any direction, the best admissible heuristic is the Euclidean distance:

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{2}$$

Since the straight line distance is the minimum distance between any two points, Euclidean distance will never underestimate the cost to reach the goal.

The edge cost of moving from one cell to another is defined as the sum of the Euclidean distance of the two cells plus a safety-aware cost function:

$$\mathcal{C}(d) = \begin{cases} M, & d < d_{\min}, \\ K\left(1 - \dfrac{d - d_{\min}}{d_{\inf} - d_{\min}}\right)^3, & d_{\min} \le d < d_{\inf}, \\ 0, & d \ge d_{\inf}. \end{cases} \tag{3}$$

where d denotes the distance from a grid cell to the nearest obstacle. The parameter $d_{\min}$ specifies a minimum safety distance: any state closer than $d_{\min}$ is assigned a very large penalty M and is thus strongly discouraged during the search. The interval $[d_{\min}, d_{\inf})$ defines a soft inflation region around obstacles, within which the cost decays smoothly to zero. The cubic term $(\cdot)^3$ produces a steep increase in cost near $d_{\min}$ while remaining relatively small close to $d_{\inf}$, so that paths are biased away from obstacle boundaries and tend to maintain a comfortable clearance. For $d \ge d_{\inf}$ the safety cost is zero, and the planner behaves like a standard A* search in free space.

Here, K is a scaling coefficient that determines the maximum penalty applied within the inflation region and controls how strongly the planner prefers paths with larger obstacle clearance.

The complete procedure executed by the planner is shown in Algorithm 1. For each expanded state, the algorithm evaluates its neighbors and augments the base motion cost with the safety-aware term $\mathcal{C}(d)$ before updating the g- and f-values in the usual A* manner.

Once the goal node has been selected from the OPEN queue, the sequence of states that forms the final path

Algorithm 1: A*

Data: Distance grid D, start s, goal g
Result: A path from s to g

Initialize $Open$, $g(s) \leftarrow 0$, $parent(s) \leftarrow s$
$f(s) \leftarrow g(s) + h(s, g)$; Insert s into $Open$
while *$Open$ not empty* **do**
  $c \leftarrow$ node in $Open$ with lowest f
  if $c = g$ **then**
    return RECONSTRUCTPATH(g,parent)
  foreach *neighbor n of c* **do**
    if *n outside grid or occupied* **then**
      continue
    $d \leftarrow D(n)$
    $cost \leftarrow \mathcal{C}(d)$
    $g_{\text{new}} \leftarrow g(c) + cost$
    if *$g(n)$ unset or $g_{new} < g(n)$* **then**
      $g(n) \leftarrow g_{\text{new}}$
      $f(n) \leftarrow g(n) + h(n, g)$
      $parent(n) \leftarrow c$
      Insert n into $Open$
return failure

must be recovered from the search tree. This is accomplished by backtracking from the goal node using the parent map constructed during the A* expansion process. The resulting procedure is summarized in Algorithm 2, which traces the parent links until the start node is reached, collects the corresponding states, and finally reverses the list to obtain the path in the correct order from start to goal.

Algorithm 2: ReconstructPath

Data: Goal node g, parent map $parent(\cdot)$
Result: Recovered path

Initialize empty list P;
$c \leftarrow g$;
while $parent(c) \neq c$ **do**
  Append c to P;
  $c \leftarrow parent(c)$;
Append start node to P;
Reverse P;
return P;

2) Exploration: Any grid cell is a frontier if it is 1) the cell is known, 2) is within the same connected component of the free space as the robot, 3) is at least `frontier search radius` away from a wall, and 4) is within a box of radius `frontier search radius` there are at least `min unknown neighbors` unknown cells.

Once all cells have been classified, we cluster frontier cells using a breadth first search (BFS). We seed the BFS with a frontier point and then search for any cells that are within `frontier cluster distance` of the seed. These are added to the BFS queue and we iterate until there are no cells in the queue. When the queue is empty, we select a new seed from the set of cells that are in no clusters and repeat until all frontier cells belong to a cluster. We then reject clusters of size less than `min cluster size` cells and return the centroids of the remaining cells.

We observe that pathing to these centroids often causes the bot to run into walls that are just outside of the known space. To combat this, we move the candidate point toward free space by setting the candidate equal to the centroid of the known free cells around the original centroid. It is possible that the candidate point lies outside of known free space. In this case, we sample alternative candidates around the candidate point and attempt to plan paths to them until a valid path is found.

D. Design and Mechanism

In this task, the robot is required to lift a $10\,\text{cm} \times 10\,\text{cm} \times 12\,\text{cm}$ block while simultaneously performing navigation and AprilTag detection. The lifting mechanism must not obstruct the LiDAR or the camera during operation. This introduces a structural challenge, as the LiDAR is mounted at a height of $20\,\text{cm}$, and the payload itself stands $12\,\text{cm}$ tall, leaving limited clearance for traditional lifting designs.

A conventional rack-and-pinion forklift requires two long guide rails mounted at the front of the robot, which would obstruct the LiDAR during operation. To avoid this, we use a scissor-lift mechanism that can adjust its collapsed height and extend vertically without occupying significant frontal space. This design keeps the LiDAR and camera unobstructed while still providing the necessary lifting range.

Torque from a servo motor drives a pinion gear, translating the rack and actuating the linkage to produce vertical lift. The mechanism operates at four height levels: pickup, transport, stacking, and final placement. Our scissor lift achieves a minimum height of $8\,\text{cm}$ and can reliably lift blocks up to $14\,\text{cm}$. Fig. 3 shows the CAD model and the final robot.

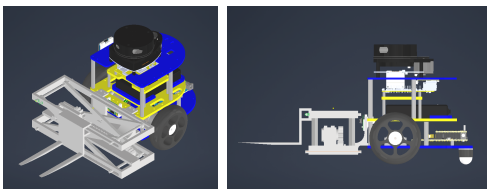


Fig. 3: CAD model: top-left and side view

E. Assembly

The assembly of the Lift Mechanism begins by securing the XL-320 servo motor (Item 9) into the motor case (Item 10), which serves as the primary drive unit. The Gear (Item 2) is then mounted onto the motor shaft to act as the pinion. For the linear motion system, the rack (Item 6) is inserted into the vertical guide channel, ensuring it meshes perfectly with the gear to facilitate smooth vertical travel as indicated in the CAD model. The structural framework is constructed using the yellow scissor arms (Item 8) to create the main tower. Finally, the lifting linkage is assembled by connecting the blue pivot pins (Item 5) arms to the tower pivots and attaching the fork (Item 7) at the distal end. The rack is coupled to this linkage system using the small plate (Item 4), allowing the servo's rotation to drive the fork mechanism upwards and downwards. The general assembly drawing of the entire MBot is shown in Fig. 5.

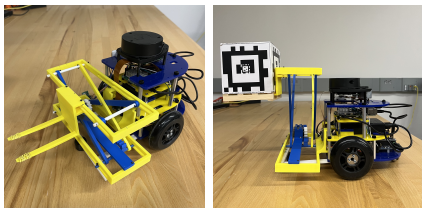


Fig. 4: Final robot: front and side view. In left picture as can be seen, rubber at the end of the fork can prevent blocks from slippery. A small ball wheel is also added at the bottom of the forklift to prevent the robot from tilting and the lift from bending after loading block weight.

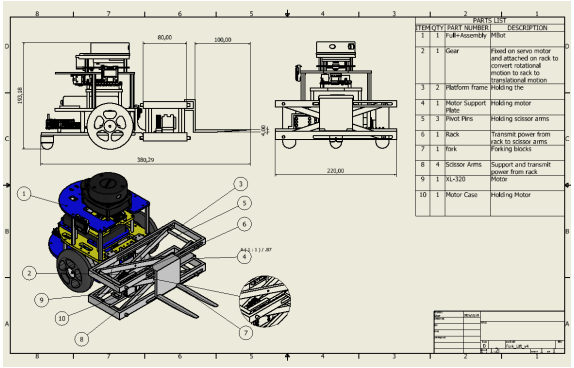


Fig. 5: General Assembly Drawing of the Entire MBot

III. Results & Discussion

A. Calibration

The standard deviations in the slope and intercepts found in the robot calibration as seen in Table I are small relative to the magnitude of the values.

The non-zero variation could come from any combination of a few sources. Noise can arise when the model does not exactly fit the phenomenon which is likely true in this case, especially if the calibration script takes data from the transient. The floor material also affects speed, so moving the robot between calibration runs will cause variation. We also can assume that there is underlying noise in the systems that generate the PWM signal and the response of the motor to the input voltage.

Since we calibrated the robot multiple times in a row without pausing between them, the position on the floor and robot geometry should not change, causing the low variances we observe. What we are likely seeing as variation is the inherent uncertainty in the physical system and the noise caused by the model not exactly fitting the real phenomenon.

Wheel	Slope	Intercept
+Left	0.055 ± 0.00048	0.052 ± 0.0078
-Left	0.0510 ± 0.00053	-0.063 ± 0.0083
+Right	0.050 ± 0.00013	0.064 ± 0.0053
-Right	0.0565 ± 0.00078	-0.060 ± 0.011

TABLE I: Wheel controller calibration. +Left means the positive direction controller for the left wheel. We report both the slope and intercept of the equation PWM $=$ $m \times$ Speed $+ b$.

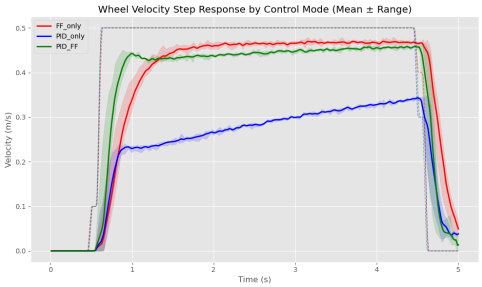


Fig. 6: Velocity response to step function of 0.5m/s. Shown is blue PID only, red is feed forward only, and green is PID and feed forward. X axis is time in seconds and y is velocity in meters per second.

B. Wheel Velocity Step Response

The combination PID and feed forward controller has the best rise time and relatively low steady state error. The PID component allows for closed loop correction to the observed velocity while the feed forward controller can quickly achieve target the desired velocity. The combination allows for a self correcting controller that rapidly achieves the desired velocity. However, none of our controllers achieved the target velocity, pointing to a potential calibration error. The addition of PID to the feed forward controller also introduces overshoot which could be mitigated by increasing the derivative term, but doing so causes the robot to back up before moving forward when the step command is given. Additionally,

increasing the integral term could reduce the steady state error further, but doing so caused the robot to fail to stop correctly when the 0 velocity command was given. This integral windup problem could be solved by resetting the accumulator when the 0 velocity command is given.

C. Simultaneous Localization and Mapping (SLAM)

In the SLAM framework, mapping and localization are tightly coupled; poor map quality reduces localization accuracy, and poor localization corrupts subsequent map updates. A major issue we encountered was the mismatch between incoming LiDAR scans and the availability of their corresponding transforms.

SLAM computation is triggered when a LiDAR scan arrives, after which the algorithm estimates and publishes the robot pose. During this process it must query the transform between `base_footprint` and `map` frame. However, when using `MovingLaserScan()` to interpolate the timestamp of the last beam, SLAM sometimes requests a transform that does not yet exist in the TF buffer—effectively asking for a pose estimate at a future timestamp that the system is currently computing. This temporal mismatch causes SLAM failure.

To solve this, we add a buffering mechanism that stores scans that cannot immediately find their corresponding transform. These scans are processed only when the required transform becomes available. This ensures that map updates use correct pose–scan associations rather than prematurely aligned data. Although this introduces a slight delay, it improves map consistency and overall localization performance.

1) Mapping and Localization: The mapping node constructs maps purely from odometry, which leads to drift over time and degraded map quality. As shown in Fig. 7, frontiers in the mapping-node output often overlap due to accumulated odometry errors. In contrast, SLAM generates higher-quality maps because it incorporates sensor measurements for localization, whereas the mapping node relies solely on odometry. Thus, odometry drift is the primary source of error in the pure mapping approach. Maze corners and walls are more clearly represented in the SLAM-generated map (right side of Fig. 7). Fig. 8 shows the estimated pose and path during localization.

Another factor affecting map quality is the way scans are interpolated. The mapping node interpolates each scan between the previous and current pose, while ideally the scan should be interpolated between the poses corresponding to the timestamps of the first and last beams. This limitation is addressed in the SLAM framework using a scan buffer, as described in following section.

The odometry can drifts slightly as the traveling distance increases. Based on the experiment, 0.5 for both

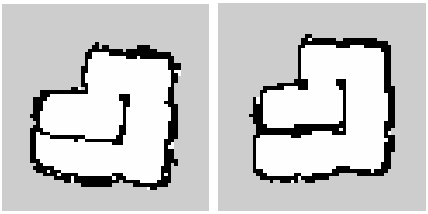


Fig. 7: Comparison between pure odometry mapping(left) and SLAM(right) with same resolution 0.05m/pixel.

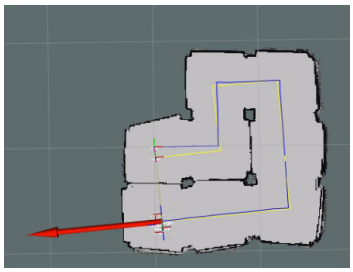


Fig. 8: Localization result. The yellow line represents the reference while the blue one represents the estimated path constructed from estimated pose.

rotational factor k1 and translational factor k2 is enough to localize the robot precisely. Larger MaxLogOdds and smaller MinLogOdds will make the map harder to flip.

a) Particle Filter: Increasing the number of particles in SLAM leads to a more diverse distribution of estimated robot poses, reducing the likelihood of particle depletion. Table II shows the computation time as a function of the number of particles, and Fig.7 illustrates that the update duration increases approximately linearly with particle count.

In our experiments, we found that using $N = 800$ particles provides sufficient precision for robot localization while maintaining a smooth SLAM update rate. Increasing the particle count to approximately $N = 1030$ causes the update frequency of the particle filter to drop below 10 Hz, which may degrade real-time performance.

Particles	Avg Time (ms)
100	12.02
500	52.70
1000	96.50

TABLE II: Update time of particle filter

b) Map Resolution: Resolution of map is the most important factor that affect SLAM quality the most, as shown in Fig. 10. Low resolution will make the robot uncertain about its location, further making the mapping worse, while high resolution increases computation burden of the robot. After testing for several values, we found a resolution with 0.01m/pixel resulting the best

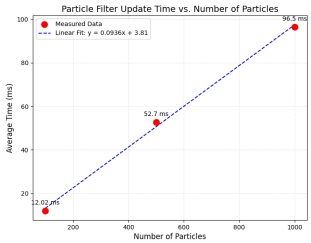


Fig. 9: Calculation time versus number of particles

map quality while not causing significant lag due to computations during SLAM.

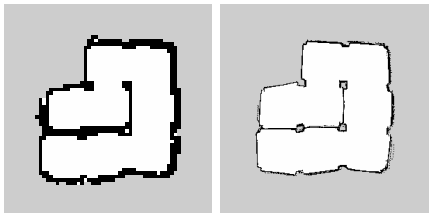


Fig. 10: SLAM with different resolution: High resolution make the robot more sure and consistent about its belief, which further improve the mapping quality.(resolution for the left one is 0.05m/pixel, 0.01m/pixel for the right one)

2) Transformation Delay: In our SLAM algorithm, the map is updated only when scans in the buffer can successfully locate their corresponding transformations with matching timestamps. Attempting to look up a transformation at $n \cdot d$ seconds after the first scan (where n is the number of rays in a scan and d is the duration of a single ray) often causes the program to crash. By using the scan buffer, each beam can find its corresponding transformation, allowing the map to be accurately constructed.

As illustrated in Fig. 11, the map on the left-hand side shows slightly less overlap at borders and along the same walls compared to the right-hand side map. Without the buffer, SLAM interpolates scans at incorrect positions, leading to overlapping features and inconsistencies in the generated map.

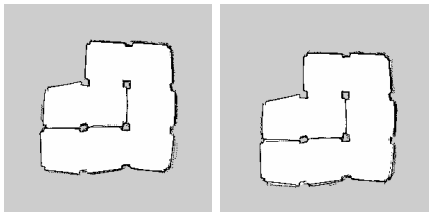


Fig. 11: SLAM with(left) and without(right) using scan buffer.

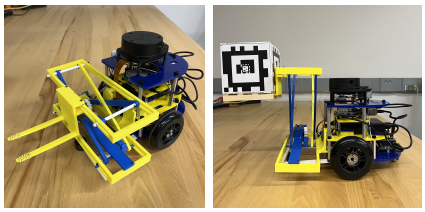


Fig. 12: Final robot: front and side view. In left picture as can be seen, rubber at the end of the fork can prevent blocks from slippery. A small ball is also added at the bottom of the forklift to prevent the robot from tilting and the lift from bending after loading block weight.

3) Initial Localization: Given an existing map we relocalize from an unknown position by implementing a new initialization phase to the particle filter. We start with 10000 particles and run just localization 10 times before resampling from 10000 to 800 particles and beginning both mapping and localization.

D. Path Planning

In this section, we present the results of our path planning experiments to evaluate the effect of incorporating the safety-aware cost function into the A* algorithm. We report two sets of results. First, we compare the planned paths obtained using the standard A* formulation and our modified A* with the distance-based safety cost. This comparison illustrates how the proposed cost function influences the trajectory, encouraging the planner to maintain greater clearance from obstacles. Second, we visualize the safety-aware cost function $\mathcal{C}(d)$ as a function of the obstacle distance d, highlighting the role of the minimum safety distance, inflation region, and cubic decay in shaping the resulting paths.

Figure 13 compares the paths produced by the standard A* and our A* that incorporates the cost function. The standard A* solution is geometrically shorter, which is indeed the optimal path in terms of path length, but it passes very close to obstacles, reflecting its tendency to minimize distance without considering clearance. In contrast, the modified A* method generates a smoother and more obstacle-averse path. The additional distance-based penalty encourages the planner to avoid high-risk regions and maintain larger clearance from obstacles, resulting in a path that is better suited for real robot navigation. This comparison highlights the effectiveness of the proposed cost function in producing safer and more conservative paths.

Table III lists the parameters used in the safety aware cost function. The minimum safety distance $d_{\min}$ defines the region where a large penalty M is applied, while the inflation radius $d_{\inf}$ specifies the range in which the cubic inflation cost is added. The coefficient K controls the

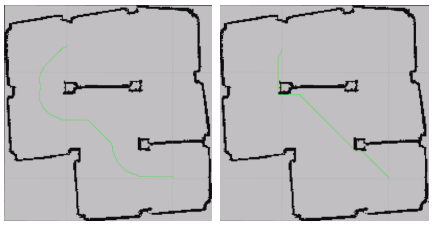


Fig. 13: Comparison of planned paths in the map with(left) and without(right) the safety-aware cost function.

magnitude of this soft penalty. These parameters together determine how strongly the planner avoids obstacles during path generation.

Parameter	Symbol	Value
Minimum safety distance	$d_{\min}$	0.15 m
Inflation radius	$d_{\inf}$	0.25 m
Maximum inflation cost	K	50.0
Hard penalty (inside $d_{\min}$)	M	1000

TABLE III: Parameters used in the safety-aware cost function.

Figure 14 illustrates the shape of the safety-aware cost function. Distances smaller than $d_{\min}$ incur a large fixed penalty, preventing the planner from entering unsafe regions near obstacles. Between $d_{\min}$ and $d_{\inf}$, the cubic decay term assigns gradually decreasing costs, encouraging the planner to maintain clearance while still allowing flexible path shaping. Beyond $d_{\inf}$, the cost is zero, and the planner behaves like standard A*. This curve shows how the cost function biases the search away from high-risk areas while preserving smooth transitions in free space.

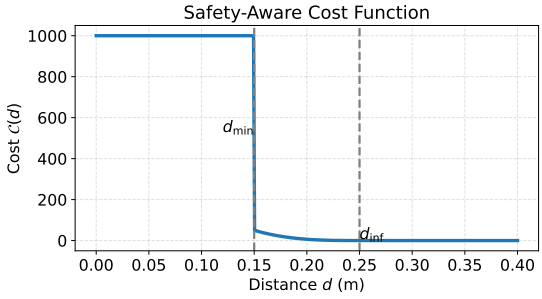


Fig. 14: Safety-aware cost function.

IV. Conclusion

This work presented an end-to-end control, planning, perception, and exploration system for the MBot. We find a hybrid turn-in-place/pure pursuit motion controller was fastest and most robust when state transitions are subject to a hysteresis mechanism. We use an occupancy-grid based simultaneous localization and mapping implementation that allows for easy planning downstream. We implemented a scan buffer to resolve the scan-transformation matching issue. We designed a scissor lift that best fits the MBot hardware structure and meets the competition requirements. We implement a safety-aware cost function for A* that ensures the MBot stays near the center of lanes and does not collide with walls. Finally, we designed a frontier exploration system that intelligently clusters frontier cells and moves the cluster centroid into free space to avoid driving too close to walls.

References

[1] J. Hu, P. Bhowmick, and A. Lanzon, "Group coordinated control of networked mobile robots with applications to object transportation," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 8, pp. 8269–8274, 2021.

[2] F. Rubio, F. Valero, and C. Llopis-Albert, "A review of mobile robots: Concepts, methods, theoretical framework, and applications," *International Journal of Advanced Robotic Systems*, vol. 16, no. 2, p. 1729881419839596, 2019.

[3] D. McNulty, A. Hennessy, M. Li, E. Armstrong, and K. M. Ryan, "A review of li-ion batteries for autonomous mobile robots: Perspectives and outlook for the future," *Journal of Power Sources*, vol. 545, p. 231943, 2022.

[4] P. Gaskell, J. Pavlasek, T. Gao, A. Narula, S. Lewis, and O. C. Jenkins, "Mbot: A modular ecosystem for scalable robotics education," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 18 294–18 300.

[5] H. Moravec and A. Elfes, "High resolution maps from wide angle sonar," in *Proceedings. 1985 IEEE international conference on robotics and automation*, vol. 2. IEEE, 1985, pp. 116–121.

[6] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte carlo localization for mobile robots," in *Proceedings 1999 IEEE international conference on robotics and automation (Cat. No. 99CH36288C)*, vol. 2. IEEE, 1999, pp. 1322–1328.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[8] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Tech. Rep., 1992.

[9] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97.'Towards New Computational Principles for Robotics and Automation'*. IEEE, 1997, pp. 146–151.

[10] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. The MIT Press, 2006. [Online]. Available: http://www.probabilistic-robotics.org/

[11] M. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2005. [Online]. Available: https://books.google.com/books?id=wGapQAAACAAJ